Féidearthachtaí as Cuimse
Infinite Possibilities

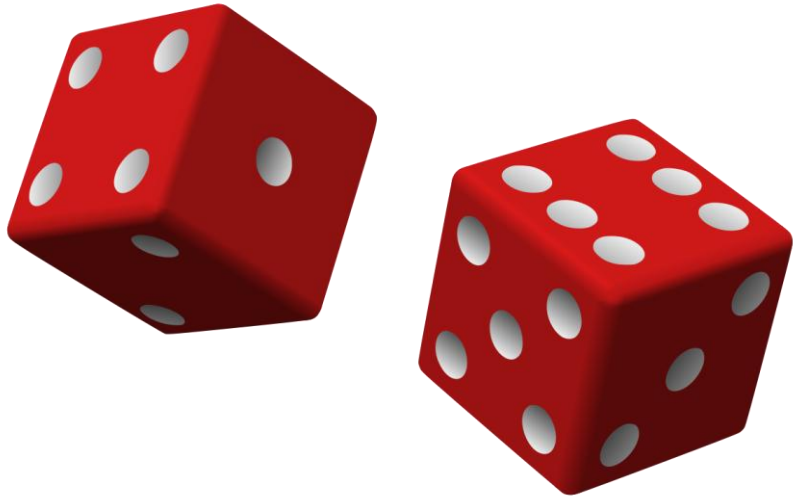
Secure Communications & Cryptography (MACS H6014)

Random Number Generators, Randomness, and One Time Pad

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TECHNOLOGICAL UNIVERSITY DUBLIN

Random Number Generators

- A simulation process in which a random component is required.
- Method of generating numbers that are random.
- Methods of generating random variates from a uniform distribution on the interval [0 1] denoted as U(0,1).
- Random variables generated from the U(0,1) distribution will be called Random Numbers.
- Casting lots, throwing dice, and dealing out cards are the best examples.
- Electronic random number indicator equipment (ERNIE) was used by the British GPO to pick winners in the lottery.

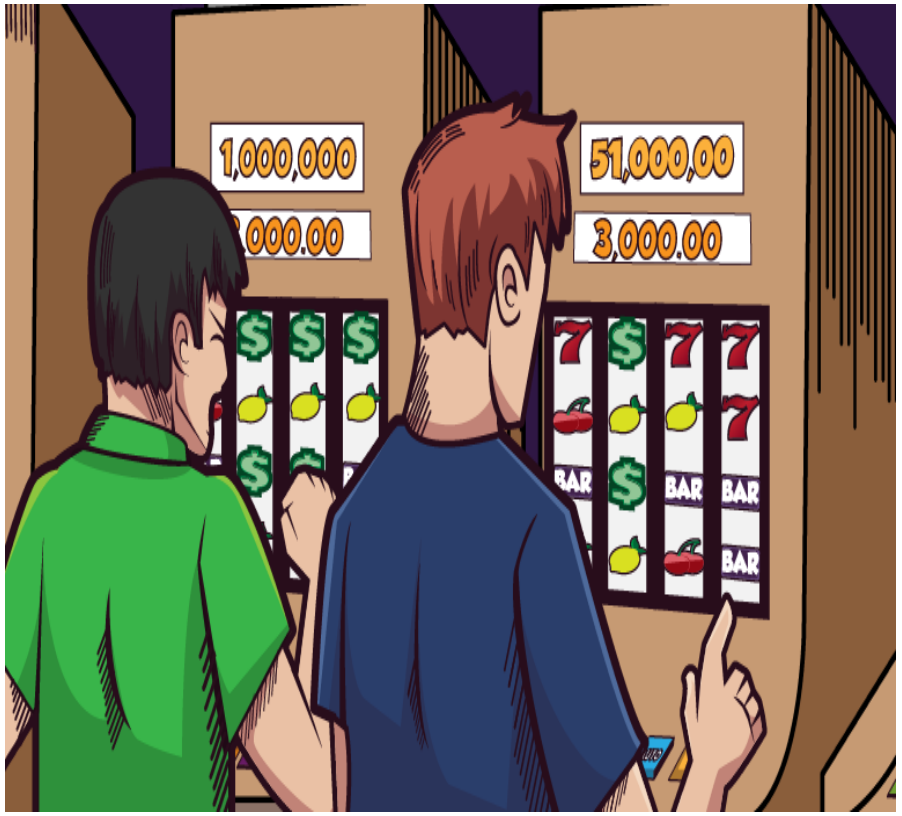


muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TU DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Types of Random Number Generators

- **Pseudo-Random Number Generators (PRNGs)**
  - Use a deterministic process to generate a sequence of numbers.
  - Generators start with an initial value and produce a sequence of numbers.
  - The generated sequence is not truly random, because totally depends on seed value.
  - PRNGs are widely used in computer programs because they are fast.
  - Common programming languages provide built-in PRNGs:
    - **'random'** module in Python
    - **'rand()'** function in C/C++
  - PRNGs are not suitable for cryptographic applications where true randomness is required.
  - Predictability makes them vulnerable to attacks



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Types of Random Number Generators (Contd.)

- **True Random Number Generators (TRNGs)**
  - Physical processes that are inherently unpredictable, such as electronic noise, radioactive decay, or atmospheric noise.
  - Entropy (unpredictable data) and convert into random numbers.
  - Used in cryptographic systems, secure key generation, and secure communications.
  - They provide a much higher level of randomness compared to PRNGs.
  - TRNGs can be implemented in hardware or software:
    - Hardware that interacts with physical processes
    - While software TRNGs use entropy sources from various inputs (e.g., mouse movements, keyboard presses) to generate

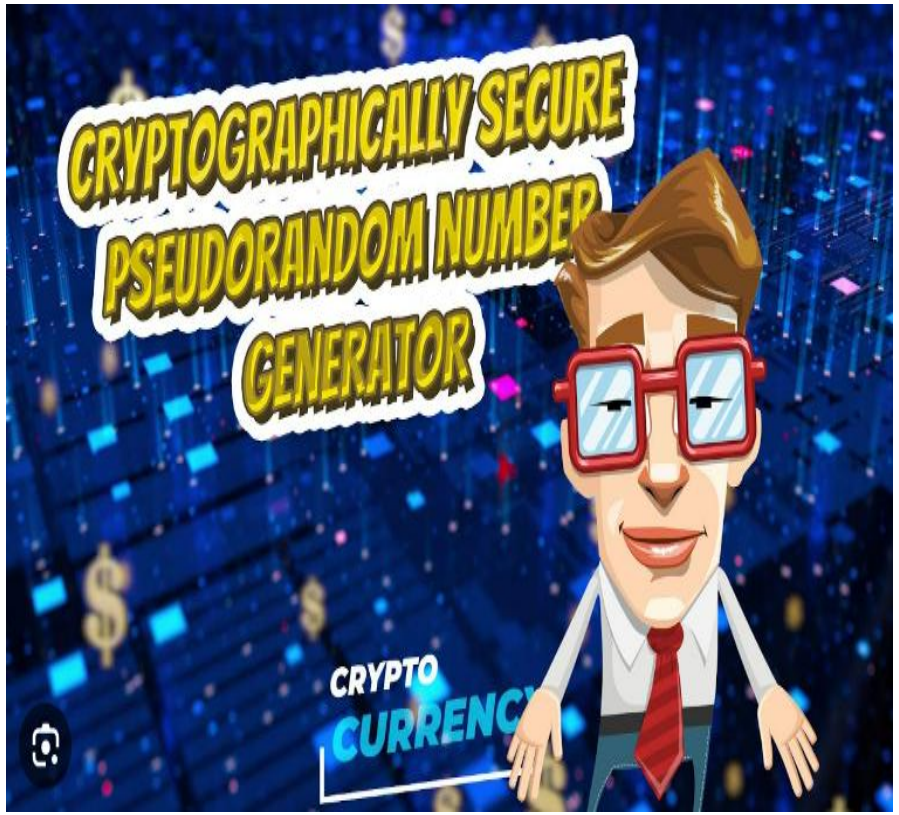


muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Types of Random Number Generators (Contd.)

- **Cryptographically Secure PRNGs (CSPRNGs)**
  - Special category of PRNGs designed to be secure for cryptographic purposes.
  - Use a combination of mathematical algorithms and entropy sources.
  - Generate random numbers that are resistant to prediction and manipulation.
  - CSPRNGs are suitable for applications like cryptographic protocols, secure key generation, password hashing, and more.
  - Key characteristics and principles are:
    - Deterministic
    - Unpredictable
    - Resistant to Predictive Attacks
    - Long Period
    - Seed Entropy and Thread Safety



muhammad.arshad@tudublin.ie

Examples of Random Number Generators

- Using Python's built-in **'random'** module to generate random numbers

```python
import random

# Generate a random integer between 1 and 100 (inclusive)
random_number = random.randint(1, 100)
print(random_number)

# Generate a random floating-point number between 0 and 1
random_float = random.random()
print(random_float)
```

- Java provides the **'java.util.Random'** class for random number generation

```java
import java.util.Random;

// Create a Random object
Random random = new Random();

// Generate a random integer between 1 and 100 (inclusive)
int randomInt = random.nextInt(100) + 1;
System.out.println(randomInt);

// Generate a random double between 0 and 1
double randomDouble = random.nextDouble();
System.out.println(randomDouble);
```

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Examples of Random Number Generators (Contd.)

- Using C++'s **'rand()'** function to generate random numbers. Note that this is a pseudo-random number generator and may require seeding with **'srand()'**

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>

int main() {
    // Seed the random number generator with the current time
    std::srand(std::time(0));

    // Generate a random integer between 1 and 100 (inclusive)
    int randomInt = std::rand() % 100 + 1;
    std::cout << randomInt << std::endl;

    // Generate a random floating-point number between 0 and 1
    double randomDouble = static_cast<double>(std::rand()) / RAND_MAX;
    std::cout << randomDouble << std::endl;

    return 0;
}
```

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Secure Online Banking Session: Example of RNGs

- **Session Key Generation (TLS Handshake)**
  - Browser establishes a TLS/SSL session, and both sides agree on a session key used to encrypt the message.
  - CSPRNG creates unpredictable session keys for encrypted communication.
  - Prevents attackers from guessing keys even after many sessions.
- **Nonce Creation in Encryption**
  - RNG generates unique nonces to ensure different ciphertexts for identical messages by the AES-GCM algorithm.
  - Protects against replay attacks and maintains message integrity.
- **One-Time Passwords (OTP) for Authentication**
  - RNG with a secret seed produces unpredictable OTPs for verification.
  - Prevents attackers from predicting authentication codes.



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Randomness

- Randomness is essential for generating secure cryptographic keys.
- Ensuring the unpredictability of cryptographic operations.
- Randomness is found everywhere in cryptography.
- Without randomness, cryptography would be impossible.
- All operations would become predictable and insecure.
- Proper random numbers shouldn't have any observable pattern.
- Randomness is characterized by a probability distribution.
- **Uniform distribution** or **Non-uniform distribution**.

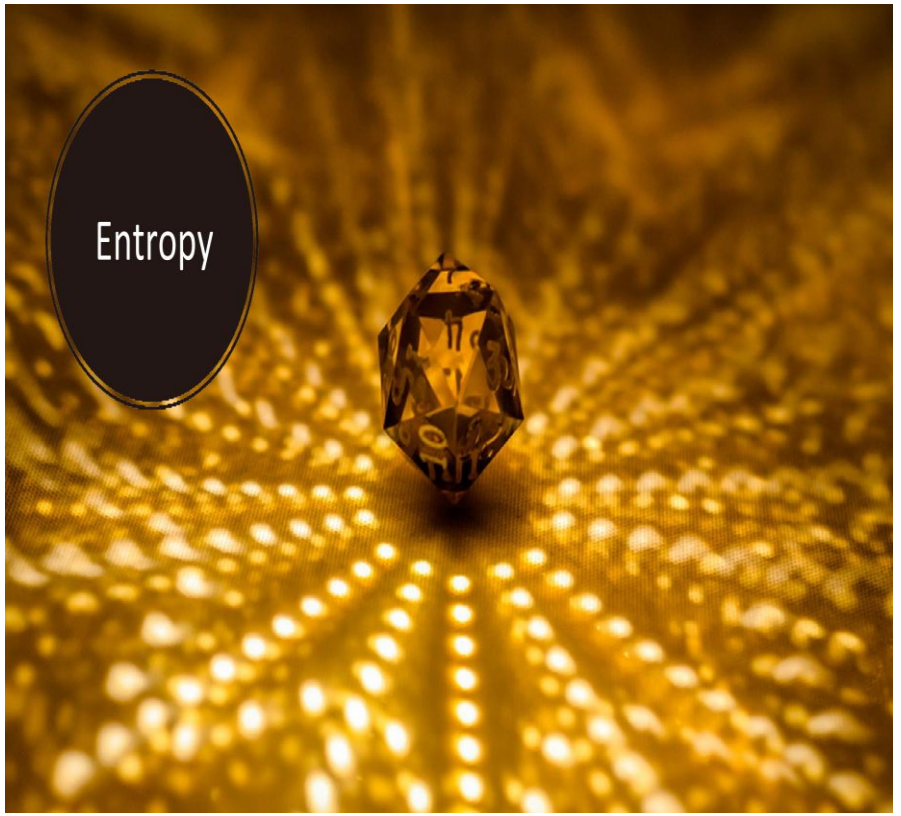


muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Entropy in Cryptography

- Entropy is a measure of the ***unpredictability*** or ***randomness*** of data.
- Generating secure cryptographic keys and ensuring the strength of cryptographic systems.
- Entropy sources:
  - Mouse movements, keyboard presses, and other user interactions.
  - Hardware Random Number Generators (HRNGs) generate true random numbers.
  - System Events are system performance statistics, hardware events, etc.
- The Entropy Pool combines and processes the input to create a more uniform and secure source of entropy.
- (Cryptographically Secure PRNGs) are designed to produce output that is highly unpredictable and resistant to prediction or bias.
- The output of the CSPRNG is used to generate cryptographic keys, Symmetric keys (AES or DES), and Asymmetric keys (public or private, RSA or ECC).



$$\sum_{n=1}^{n255} fn \ log\,2\,(fn)$$

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Randomness in Cryptography

- Key Generation
  - Cryptographic keys serve as the foundation of most cryptographic operations.
  - Randomness is used to generate cryptographic keys.
- Initialization Vectors (IVs)
  - Used to add randomness to the encryption process, making it more secure.
- Salts and Nonces
  - Introduced to add randomness and prevent precomputed attacks.
- Random Number Generators (RNGs)
  - CSPRNGs are used to generate random keys, initialization vectors, salts, nonces, and other cryptographic parameters.



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Randomness in Cryptography (Contd.)

- Digital Signatures
  - RSA or DSA algorithms in signatures prevent attackers from predicting the signature and forging it.
- Session Keys and Tokens
  - To ensure confidentiality, used in authentication and authorization.
- Cryptographic Protocols
  - SSL/TLS for secure web communication or IPsec for secure network communication.
  - To establish secure connections and protect data during transmission.
- Entropy Sources
  - Measure of randomness, to seed their random number generators.
  - Can be obtained from various sources e.g. mouse movement, keyboard presses, etc.



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Sending a Secure WhatsApp Message: Example of Randomness

- **Key Exchange (End-to-End Setup)**
  - Apps run the Signal Protocol.
  - Randomness generates ephemeral key pairs (private/public).
  - Ensures unique sessions, blocking key reuse by attackers.
- **Message Encryption (Forward Secrecy)**
  - App uses a random value (nonce + ephemeral keys) in AES or ChaCha20.
  - Random nonces make ciphertexts always different.
  - Prevents pattern analysis across repeated messages.
- **Authentication & Verification**
  - Random codes used for identity checks (QR/number).
  - Stops attackers from forging or predicting tokens.



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Encoding Methods

- Binary
- Hex
- ASCII
- Base64

With Base-64, the bits are split into groups of six, and then converted. Base-64 is used extensively on the Internet (such as in email).

'D' is 68 (decimal) in ASCII table

The binary code for the letter 'D' is 01000100.

The hex value is 44. Hex can be represented in several ways: base 16 e.g. 44_{16}

or 0x44 or 44hex or 44h

$0x44 = 44_{16} = 01000100_2 = 0 + 2^6 + 0 + 0 + 0 + 2^2 + 0 + 0 = 64_{10} + 4_{10} = 68_{10}$

Decimal	Binary	Hex
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

ASCII Characters

- ASCII (American Standard Code for Information Interchange) is a one-byte code used to represent characters. Standard ASCII uses the low 7 bits and can represent 128 (2 7 characters.
- The first 33 numbers (000 to 032) in the ASCII table are unprintable control codes and are used to control peripherals such as printers.

Regular ASCII Chart (character codes 0 - 127)

000		(nul)	016	►	(dle)	032	sp	048	0	064	@	080	P	096	`	112	p
001	☺	(soh)	017	◄	(dc1)	033	!	049	1	065	A	081	Q	097	a	113	q
002	☻	(stx)	018	↕	(dc2)	034	"	050	2	066	B	082	R	098	b	114	r
003	♥	(etx)	019	‼	(dc3)	035	#	051	3	067	C	083	S	099	c	115	s
004	♦	(eot)	020	¶	(dc4)	036	$	052	4	068	D	084	T	100	d	116	t
005	♣	(enq)	021	§	(nak)	037	%	053	5	069	E	085	U	101	e	117	u
006	♠	(ack)	022	▬	(syn)	038	&	054	6	070	F	086	V	102	f	118	v
007	•	(bel)	023	↨	(etb)	039	'	055	7	071	G	087	W	103	g	119	w
008	◘	(bs)	024	↑	(can)	040	(	056	8	072	H	088	X	104	h	120	x
009		(tab)	025	↓	(em)	041	)	057	9	073	I	089	Y	105	i	121	y
010		(lf)	026		(eof)	042	*	058	:	074	J	090	Z	106	j	122	z
011	♂	(vt)	027	←	(esc)	043	+	059	;	075	K	091	[	107	k	123	{
012	♀	(np)	028	∟	(fs)	044	,	060	<	076	L	092	\	108	l	124	|
013		(cr)	029	↔	(gs)	045	-	061	=	077	M	093	]	109	m	125	}
014	♫	(so)	030	▲	(rs)	046	.	062	>	078	N	094	^	110	n	126	~
015	☼	(si)	031	▼	(us)	047	/	063	?	079	O	095	_	111	o	127	⌂

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Brief History of ASCII Code

- The American Standard Code for Information Interchange, or ASCII code, was created in 1963 by the "American National Standards Institute" (ANSI).
- (ASCII/ANSI codeset was developed by reordering and expanding the set of symbols and characters already used in telegraphy at that time by the Bell company.
- At first, only included capital letters and numbers but in 1967 was added lowercase letters and some control characters, forming what was known as US ASCII, i.e. the characters 0 through 127.
- This set of only 128 characters was published in 1967 as standard, containing all you need to write in English language.
- In 1981 IBM developed an extension of 8-bit ASCII code, called ***"code page 437"***, in this version, some obsolete control characters for graphic characters.
- Also, 128 characters were added with new symbols, signs, graphics, and Latin letters, all punctuation signs, and characters needed to write texts in other languages, such as Spanish In this way was added the ASCII characters ranging from 128 to 255.
- IBM includes support for this code page in the hardware of its model 5150 known as "IBM PC", considered the first personal computer.
- The operating system of this model, the ***"MS-DOS"*** also used this extended ASCII code Almost all computer systems today use the ASCII code to represent characters and texts.

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Extend ASCII Code

Extended ASCII means an **eight-bit character encoding** that includes the seven-bit **ASCII** characters, plus additional characters. There are many extended ASCII encodings (more than 220 DOS and Windows codepages). **EBCDIC** ("the major 8-bit character code from IBM) likewise developed many extended variants (more than 186 EBCDIC codepages) over the decades. Mainly used on IBM mainframes and mid-size computers.

128	Ç	144	É	160	á	176	▒	193	┴	209	╤	225	ß	241	±
129	ü	145	æ	161	í	177	▒	194	┬	210	╥	226	Γ	242	≥
130	é	146	Æ	162	ó	178	▓	195	├	211	╙	227	π	243	≤
131	â	147	ô	163	ú	179	|	196	─	212	╘	228	Σ	244	⌠
132	ä	148	ö	164	ñ	180	┤	197	┼	213	╒	229	σ	245	⌡
133	à	149	ò	165	Ñ	181	╡	198	╞	214	╓	230	µ	246	÷
134	å	150	û	166	ª	182	╢	199	╟	215	╫	231	τ	247	≈
135	ç	151	ù	167	º	183	╖	200	╚	216	╪	232	Φ	248	°
136	ê	152	_	168	¿	184	╕	201	╔	217	┘	233	⊕	249	·
137	ë	153	Ö	169	_	185	╣	202	╩	218	┌	234	Ω	250	·
138	è	154	Ü	170	¬	186	╢	203	╦	219	▉	235	δ	251	√
139	ï	156	£	171	½	187	╗	204	╠	220	▄	236	∞	252	_
140	î	157	¥	172	¼	188	╝	205	═	221	▌	237	ϕ	253	²
141	ì	158	_	173	¡	189	╜	206	╬	222	▐	238	ε	254	■
142	Ä	159	ƒ	174	«	190	╛	207	╧	223	▀	239	∩	255	
143	Å	192	└	175	»	191	┐	208	╨	224	α	240	≡		

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Base64

- Each **Base64** digit represents exactly 6 bits of data.
- Three 8-bit bytes (i. a total of 24- -bits) can therefore be represented by four 6-bit Base64 digits.
- Base64 is designed to carry data stored in binary formats across channels that only reliably support text content.
- Base64 is used on the Internet to
- embed image files or other binary assets inside textual assets such as HTML and CSS files.
- Base64 is also widely used for sending e-mail attachments.

Index	Binary	Char	Index	Binary	Char	Index	Binary	Char	Index	Binary	Char
0	000000	A	16	010000	Q	32	100000	g	48	110000	w
1	000001	B	17	010001	R	33	100001	h	49	110001	x
2	000010	C	18	010010	S	34	100010	i	50	110010	y
3	000011	D	19	010011	T	35	100011	j	51	110011	z
4	000100	E	20	010100	U	36	100100	k	52	110100	0
5	000101	F	21	010101	V	37	100101	l	53	110101	1
6	000110	G	22	010110	W	38	100110	m	54	110110	2
7	000111	H	23	010111	X	39	100111	n	55	110111	3
8	001000	I	24	011000	Y	40	101000	o	56	111000	4
9	001001	J	25	011001	Z	41	101001	p	57	111001	5
10	001010	K	26	011010	a	42	101010	q	58	111010	6
11	001011	L	27	011011	b	43	101011	r	59	111011	7
12	001100	M	28	011100	c	44	101100	s	60	111100	8
13	001101	N	29	011101	d	45	101101	t	61	111101	9
14	001110	O	30	011110	e	46	101110	u	62	111110	+
15	001111	P	31	011111	f	47	101111	v	63	111111	/
Padding		=									

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Example of ASCII to Base64

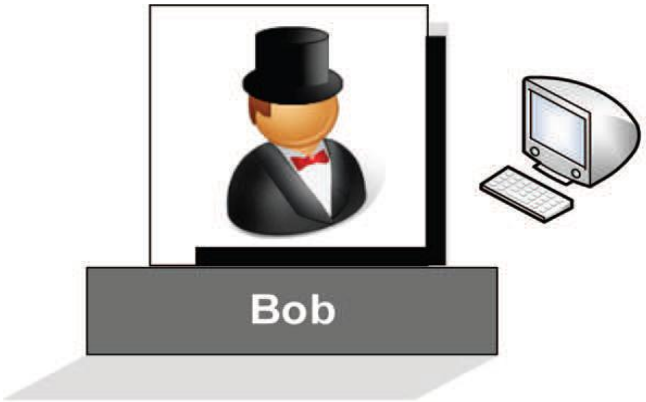
The encoded value of ***Man*** is ***TWFu*** Encoded in ASCII, the characters *M*, *a*, and *n* are stored as the byte values 77, 97, and 110, which are the 8-bit binary values 01001101 01100001, and 01101110. These three values are joined together into a 24-bit string, producing 010011010110000101101110. Groups of (6 bits have a maximum of 2^6 = 64 different binary values) are converted into individual numbers from left to right (in this case, there are four numbers in a 24-bit string), which are then converted into their corresponding Base 64 character values.

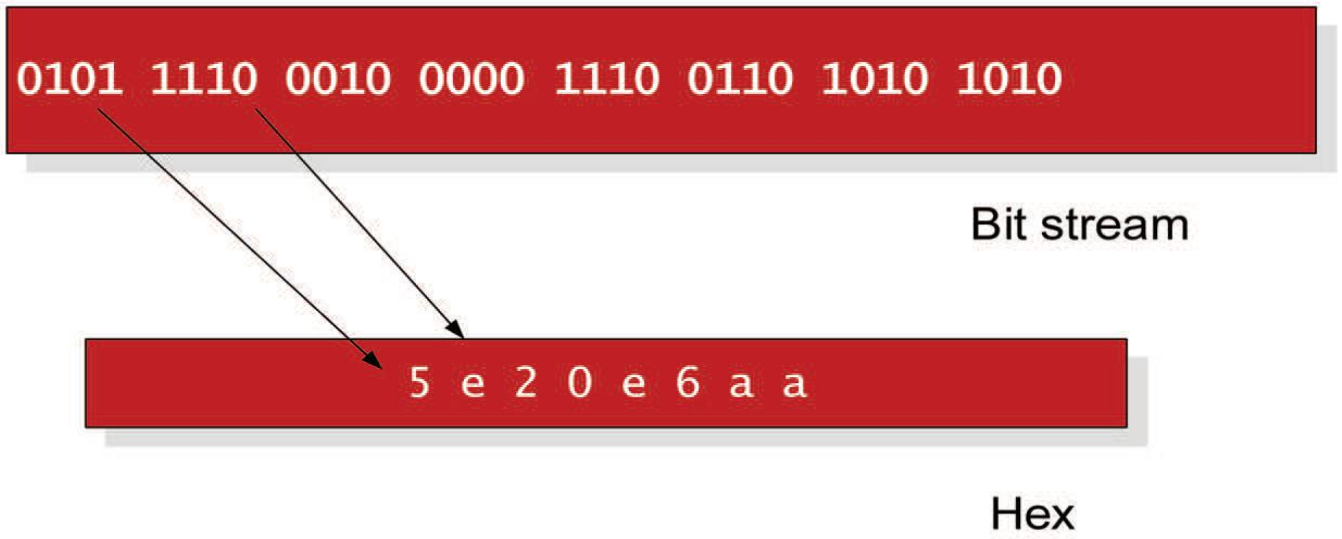
Source	**Text (ASCII)**	**M**								**a**								**n**							
	Octets	77 (0x4d)								97 (0x61)								110 (0x6e)							
Bits		0	1	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0	1	1	0	1	1	1	0
Base64 encoded	**Sextets**	19								22				5				46							
	Character	**T**								**W**				**F**				**u**							
	Octets	84 (0x54)								87 (0x57)				70 (0x46)				117 (0x75)							

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Conversion to Hex



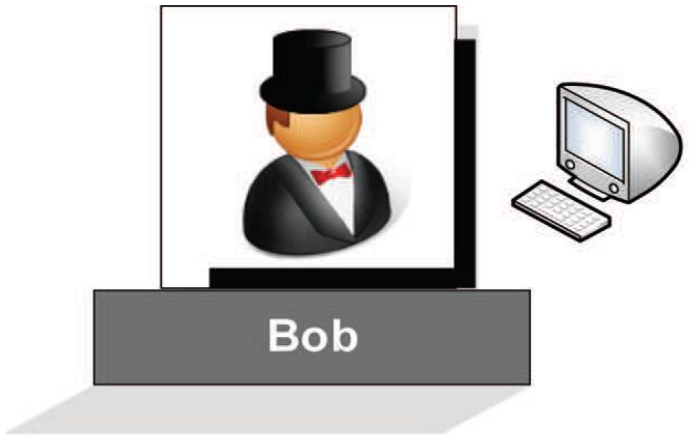


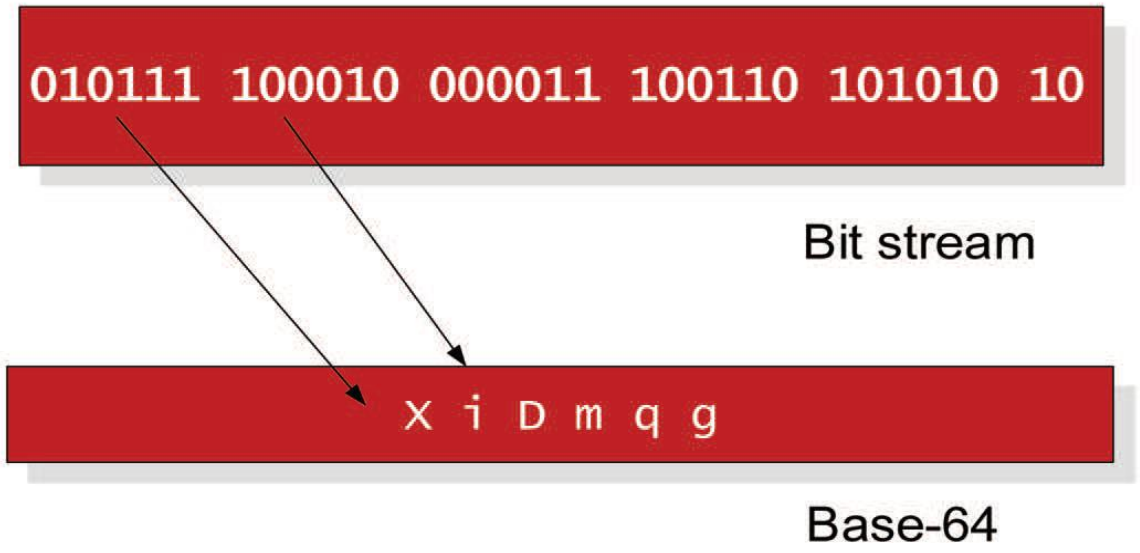
Decimal	Binary	Hex
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Conversion to Base-64





Val	Enc	Val	Enc	Val	Enc	Val	Enc
0	A	16	Q	32	g	48	w
1	B	17	R	33	h	49	x
2	C	18	S	34	i	50	y
3	D	19	T	35	j	51	z
4	E	20	U	36	k	52	0
5	F	21	V	37	l	53	1
6	G	22	W	38	m	54	2
7	H	23	X	39	n	55	3
8	I	24	Y	40	o	56	4
9	J	25	Z	41	p	57	5
10	K	26	a	42	q	58	6
11	L	27	b	43	r	59	7
12	M	28	c	44	s	60	8
13	N	29	d	45	t	61	9
14	O	30	e	46	u	62	+
15	P	31	f	47	v	63	/

muhammad.arshad@tudublin.ie

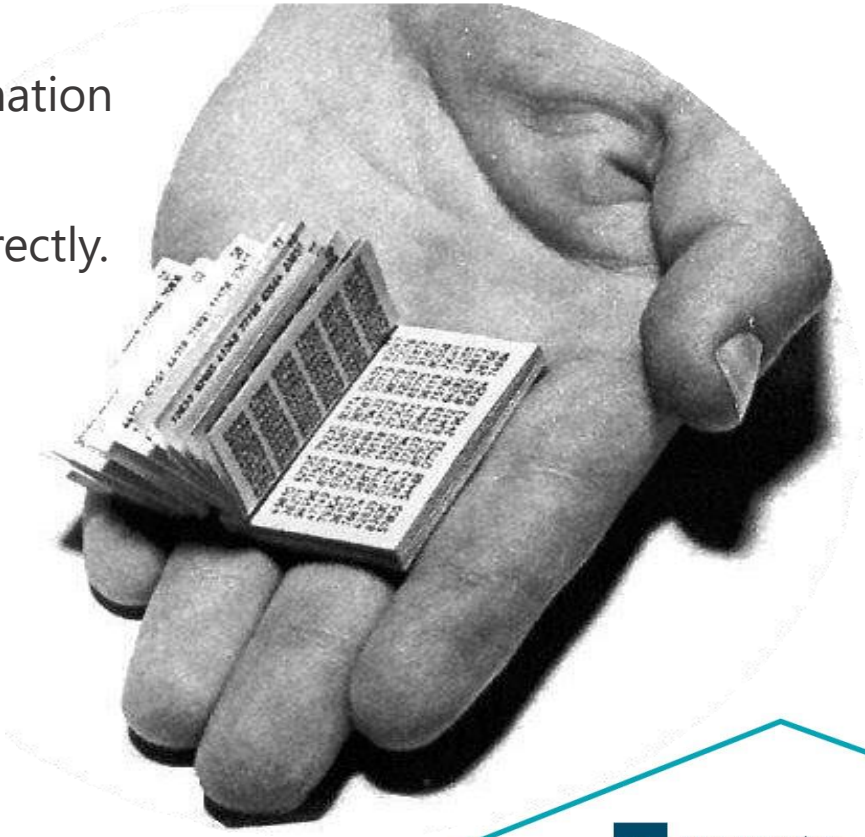
OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

One Time Pad

- Very simple encryption algorithm based on modular arithmetic.
- Each key must be completely random and used only once.
- OTP is perfectly secure because the cipher text provides no information about the plaintext other than the maximum possible size.
- **"Perfect cryptography"** – cannot be cracked if implemented correctly.
- OTP Formula:

$c = E(k, m) = k \oplus m$
$m = D(k, c) = k \oplus c$

msg: 0 1 1 0 1 1 1
key: 1 0 1 1 0 1 0 $\oplus$
CT: 1 1 0 1 1 0 1



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

How Does OTP Work?

- Truly random keys are transferred beforehand [often in bulk in the form of a “pad”].
- The sender creates the ciphertext by using modular arithmetic on the plaintext.
- And key a character at a time in order to calculate the corresponding character in the cipher text. (Bitwise XOR on the data)
- The sender immediately destroys his copy of the key and then sends the ciphertext by any means to the receiver.
- The receiver performs the same modular arithmetic with the ciphertext and key.
- The receiver then destroys his copy of the key.

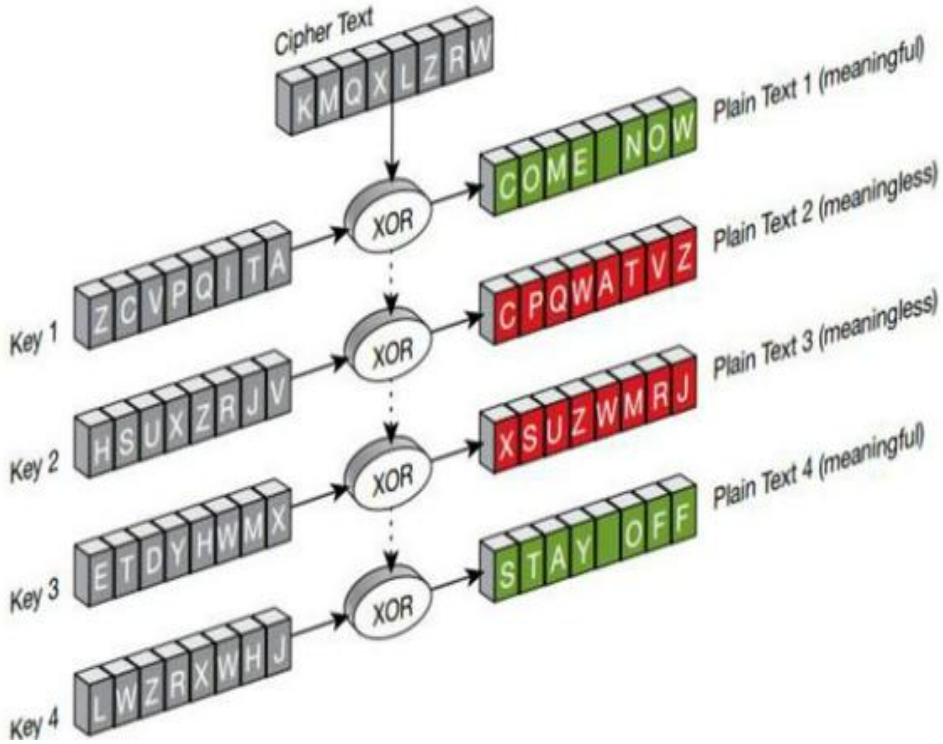


muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Why is it perfectly secure?

- Let's say we want to brute force (trial-and-error) attack the one-time pad.
- This means:
  - Guess every single key possible until we get the right key.
  - Then XOR the ciphertext with the key to get the plaintext.
- ***Impossible to know which key yielded the actual message.***
- Brute forcing the key will never work, because
- We have no idea whether you brute-forced the right key.
- The receiver then destroys his copy of the key.



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

OTP Example

- Step 1: Key Generation
  - Let's generate a random key of the same length as the message. Our message is "HELLO," which we'll represent in binary:
  - **Message: 01001000 01000101 01001100 01001100 01001111** (ASCII representation of "HELLO")
  - Now, we generate a random key of the same length:
  - **Key: 11010110 00110111 10001100 11100100 10101010**



- Step 2: Encryption
  - To encrypt the message, we perform an XOR operation between the message and the key:
  - Ciphertext = Message XOR Key

```
Ciphertext: 10011110 01110010 11000100 00101000 01100101
```

  - So, the ciphertext is 10011110 01110010 11000100 00101000 01100101.
- Step 3: Decryption
  - To decrypt the ciphertext and recover the original message, we use the same key and perform an XOR operation:
  - Decrypted Message = Ciphertext XOR Key

```
Decrypted Message: 01001000 01000101 01001100 01001100 01001111
```

  - This decrypted binary data corresponds to the ASCII representation of "HELLO," which is the original message.

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Attacks Against OTP

OTP is theoretically secure but has vulnerabilities because of practical implementation and human errors. Here are some potential attacks and weaknesses:

- **Key Reuse**
  - If the same key is ever used to encrypt more than one message.
- **Key Generation**
  - If the key generation process is flawed or not truly random.
- **Key Distribution**
  - If an attacker intercepts the key during the distribution.
- **Key Management**
  - If keys are mishandled or compromised.
- **Entropy Source**
  - If the source is biased or predictable.
- **One-Time Pad Length**
  - Maintaining keys that are as long as the message.
- **Human Error**
  - Human errors, such as reusing keys, failing to destroy used key sheets, or choosing easily guessable keys.
- **Physical Security**
  - Physical theft of key materials or devices can compromise security.

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Military Communication in a Warzone: Example of OTP

- **Setup (Key Distribution)**
  - Two field units, Alpha and Bravo, need to exchange top-secret coordinates.
  - Alpha and Bravo receive identical random keypads.
  - Each page is used only once for secrecy.
- **Encryption (Sender – Alpha)**
  - Alpha wants to send the message: **“ATTACK AT DAWN.”**
  - Message converted to numbers and XORed with the key.
  - Ciphertext appears completely random and unbreakable.
- **Decryption (Receiver – Bravo)**
  - Ciphertext XORed with the same key to recover the message.
  - True randomness and a single-use key ensure perfect security.



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TU DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Short Quiz

- **A random number generator (RNG) produces a sequence of integers between 1 and 10, inclusive. What is the probability that it will generate the number 7 on a single trial??**
  - Since the RNG generates integers between 1 and 10, inclusive, there are 10 possible outcomes (numbers 1 through 10). The probability of generating the number 7 on a single trial is 1/10 or 10%.
- **Encrypt the "hello word" by OTP, and the key is "I not know"?**

```
01101000 01100101 01101100 01101100 01101111 00100000 01110111 01101111 01110010 01100100
XOR
01001001 00100000 01101110 01101111 01110100 00100000 01101011 01101110 01101111 01110111
=
00100001 01000101 00000010 00000011 00011011 00000000 00011100 00000001 00011101 00010011
```

muhammad.arshad@tudublin.ie

Practical Exercise

- **Week 3 – 2nd Oct**
  - Complete **Lab 1A** - Ciphers and Fundamentals worksheet
  - Complete **Lab 1B** - Numbers station
  - Lab 1C – **Many Time Pad (Challenge Lab)** due on **4th October**.
  - Read **Chapters 1 & 2** – Ciphers and Fundamentals & Secret Key Encryption - Cryptography by William Buchanan
  - Optional - Attempt some **extra challenges** on Cryptohack.org
  - **Test 1** on **23rd October** during the lab session

muhammad.arshad@tudublin.ie